

实验报告

课程名称: Design pattern and software architecture

学 院: 计算机科学与工程学院

专 业: Computer Science and 班 级: SE-1

Technology

姓 名: Muhammad Samudera 学 号:

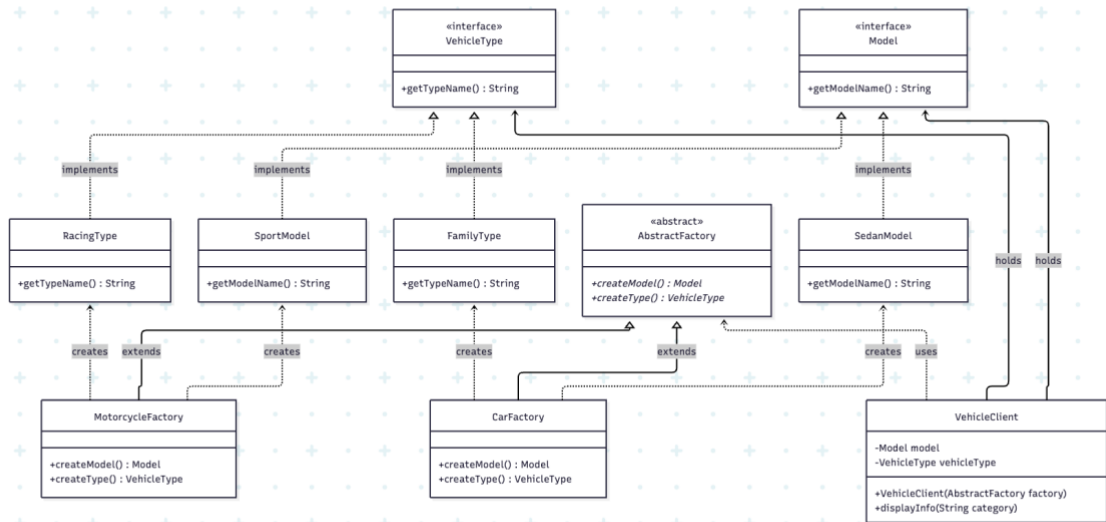
年 月 日

山东科技大学教务处制

实 验 报 告

_____页

组 别	1	姓 名	Samudera	同组实验者	None
实验项目名称	Abstract Factory Pattern				
教师评语	Good understanding of the pattern idea and implementation. The report is clear and written in proper English.				
实验成绩:			指导教师（签名）： 2026 3 23		
1. Objective Abstract Factory design pattern using a vehicle example. The goal of this pattern is to create related objects without specifying their exact concrete classes in the client code. In this example, the program has two product families: - `Model` - `VehicleType` Each factory creates one model and one vehicle type that belong to the same family. 2. Principle Abstract Factory provides an interface for creating families of related objects without specifying their concrete classes. The client only depends on abstractions, so the product family can be switched easily at runtime. 3. UML Diagram					



4. Key Code (Java)

```

interface Model {

    String getModelName();

}

interface VehicleType {

    String getTypeName();

}

abstract class AbstractFactory {

    abstract Model createModel();

    abstract VehicleType createType();

}

```

```
}
```

```
class CarFactory extends AbstractFactory {
```

```
    @Override
```

```
    Model createModel() {
```

```
        return new SedanModel();
```

```
    }
```

```
    @Override
```

```
    VehicleType createType() {
```

```
        return new FamilyType();
```

```
    }
```

```
}
```

```
class MotorcycleFactory extends AbstractFactory {
```

```
    @Override
```

```
    Model createModel() {
```

```
        return new SportModel();
```

```
    }
```

```
    @Override
```

```
    VehicleType createType() {
```

```

        return new RacingType();
    }
}

class VehicleClient {

    private final Model model;

    private final VehicleType vehicleType;

    VehicleClient(AbstractFactory factory) {

        this.model = factory.createModel();

        this.vehicleType = factory.createType();

    }

    void displayInfo(String category) {

        System.out.println("Category : " + category);

        System.out.println("Model      : " + model.getModelName());

        System.out.println("Type       : " + vehicleType.getTypeName());

        System.out.println();

    }

}

```

5. Analysis

Advantages: It guarantees compatibility between related products, reduces coupling between the client and concrete classes, and makes it easy to replace an entire product family.

Disadvantage: Adding a new product type requires changing the abstract factory interface and every concrete factory implementation.

6. Conclusion

Abstract Factory is suitable for systems that need to support multiple related product families, such as cars and motorcycles in this example. In this program, the client does not create objects directly. Instead, it relies on the abstract factory interface to obtain a complete set of related products, which keeps the code clean and easy to understand.

This approach separates object creation from object usage, so the client code remains independent from concrete classes. As a result, changes in one product family do not affect the client as long as the factory interface stays the same. This makes the design easier to maintain, especially when the application grows and requires more product variants in the future.

The pattern also improves consistency because each factory is responsible for creating objects that belong together. In the vehicle example, `CarFactory` always returns car-related products, while `MotorcycleFactory` always returns motorcycle-related products. This guarantees that the created objects are compatible and follow the intended design.

Overall, Abstract Factory is a strong choice when an application must support interchangeable product families while keeping creation logic organized and flexible. It provides a scalable structure that is easy to extend, test, and maintain.

